Assignment 3 — Clustered Forward Shading

Advanced Computer Graphics, 5DV180

Olle Lögdahl
olle.logdahl@umu.se

I. Introduction

The following document describes the design of a clustered forward shading renderer, backed by GPU-driven rendering. The new renderer supports many lights (up to 10′000 in real time).

II. Usage guide

In the following section, the usage of the program is described. Firstly in Section II.A, the building process is described.

II.A. Building and running

For building the application, the following are required:

- C++20 / C++2a compatible compiler.
- `GLFW` installed system-wide.
- `make` or `cmake` (and a buildsystem like `make` or `ninja`)

The following `make` commands are available:

- `make dev` (default) - Builds the program in dev mode. Full optimizations but with tracing enabled.
- `make release` - Builds the program in release mode.
- `make memcheck` - Builds the program with ubsan, asan and validation layers enabled.
- `make clean` - Cleans the build directory.

After building the program, it can be run with `./cgr <scene file>`. The program requires `glslc` [1] to be installed and on `$PATH`, as it is used to compile glsl shaders. If `glslc` is not found, the path can be specified using the `GLSLC_PATH` environment variable at runtime.

The project can also be built using cmake:

- `cmake -B build -DCMAKE_BUILD_TYPE=Debug`
- `cmake --build build --parallel`

II.B. System Requirements

Below are the system requirements for running the application. Note that the program only works on linux curretly.

- Vulkan 1.3 compatible GPU. The descriptor indexing feature is also required. These are generally widely supported.

III. Background

A classical forward renderer suffers from some issues, the biggest usually being overdraw. An overdrawn fragment is both wasted work, as well as inefficient usage of fill rate. As each fragment is being rendered in a forward-fashion, costly lighting calcuation occurs for fragments which will not be seen. Historically, deferred rendering has been proposed to solve this. By rendering only albedo, normals and other properties per-object, and later shading by pixel, the shading is only performed on every visible fragment. While this solves overdraw, deferred shading comes with other issues relating to bandwidth as well as anti-aliasing.

The issue of overdraw can be handled in many ways, like performing an early depth-only pass or by triangle reordering. [2] In this assignment, we focused on lessening the overdraw impact by reducing the cost of fragment shading. Our method, Clustered Shading, can also efficiently be applied to a deferred renderer to support many lights. Our solution is capable of rendering a scene with 10′000 point lights in real time.

IV. Clustered Forward Shading

Clustered shading is a method to reduce lighting calculations by reducing the number of lights that are tested against each fragment. [3] The method works by splitting the view frustum into frustum shaped boxes called clusters and assigning the lights to the clusters based on their influence radius. When shading, each fragment can look up the containing cluster and only calculate shading using those lights. This means that clusters with no lights perform no lighting calculations.

In our implementation, we define the clusters using axis-aligned bounding boxes in view space. This is not completely accurate, as there will be slight overlap, but this doesn't cause any issues in the tested scenes.

A great feature of clustering is that the system can be reused for other spatial-related systems like decals and light probes. [4]

IV.A. Cluster Generation

The first step is to generate the clusters bounding boxes. The clusters needs to be recalculated every time the projection changes. For clustering, we define a grid size for the number of clusters in the frustum; we use the same as [4] which is 16x8x24. Each compute invocation calculates the bounding box for a single cluster, using its invocation ID to calculate the cluster position in view space.

While the X and Y axis are divided into equal parts, the Z axis is sliced logarithmically to counteract both the fact that the NDC space is not linear and that clusters closer to the camera likely cover more fragments. The start depth of slice n (Z_n) is

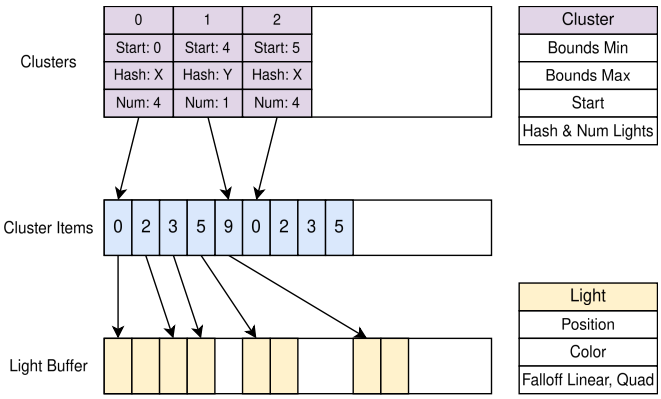

Figure 1: Memory layout of clusters, cluster items and lights.

calculated using (1), where N is the total number of Z slices. [4]

$$Z_n = Z_{\text{near}} * \left(\frac{Z_{\text{far}}}{Z_{\text{near}}} \right)^{\frac{n}{N}} \quad (1)$$

This gives us a cheap mapping between view-space position and cluster index.

IV.B. Cluster Assignment

Assigning lights to clusters is the main part of the clustering algorithm. In previous assignments we already store all point lights in a buffer, so we use an indirection buffer similar to [4] to reference them. After this pass, each cluster will have a reference to the start of the cluster items, as well as a hash and the number of lights. Each cluster supports up to 255 lights. See Figure 1 for a memory layout of clusters, cluster items and lights.

Assignment is done using a simple AABB-sphere intersection test. As the lights don't store their radii, this needs to be calculated using the intensity and the falloff parameters. Using a naive approach, we were able to assign 10′000 lights to 2′800 clusters in 6ms on a GTX 1070. While an extreme case, we optimized the algorithm to perform faster and require less memory.

The naive approach would be to let each invocation handle a single cluster, and then iterate over all lights directly. This requires alot of memory accesses (although they are mostly coherent), and repeats light radii calculations. Another issue is that the amount of memory required is very high, as there is

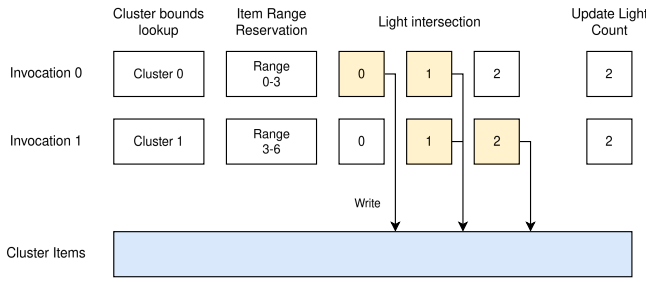

Figure 2: Naive cluster assignment implementation.

no communication between clusters. Therefore, each cluster needs to reserve a worst-case amount of memory. If using a 16x8x24 grid, this would require 3.1MB of memory. Also consider that when looking up items during shading, a sparse lookup would occur in the items buffer. This implementation is shown in Figure 2, simplified for 3 lights and 2 clusters.

We propose an optimized approach using shared memory and subgroup operations. The cluster assignment is broken up into multiple stages; prefetching, intersection, and writing. The stages require explicit barriers to ensure that all invocations are in sync. The optimized approach uses light batches, with the same batch size as the size of the workgroup. First, each invocation collaborates on reading a single light of the following batch into shared memory. After this, the radii have been calculated and all lights. The intersection tests work as before, still using coherent reads, but from shared memory. Each invocation keeps a running hash of the light IDs that intersected, as well as store the lights into invocation-private memory. After there are no more batches to process, the invocations hashes are compared. If multiple invocations have the same hash, the one with the lowest ID writes the lights from private memory to the cluster list, and atomically increments the number of items in the list. This implementation is shown in Figure 3, simplified for 5 lights and 4 clusters.

The optimized implementation using the batch system took the time to assign 10′000 lights to 2′800 clusters down to 1.3ms on a GTX 1070. Compressing the cluster items reduced this time down to 1.1ms. Importantly, the memory usage also went down to 164000 bytes.

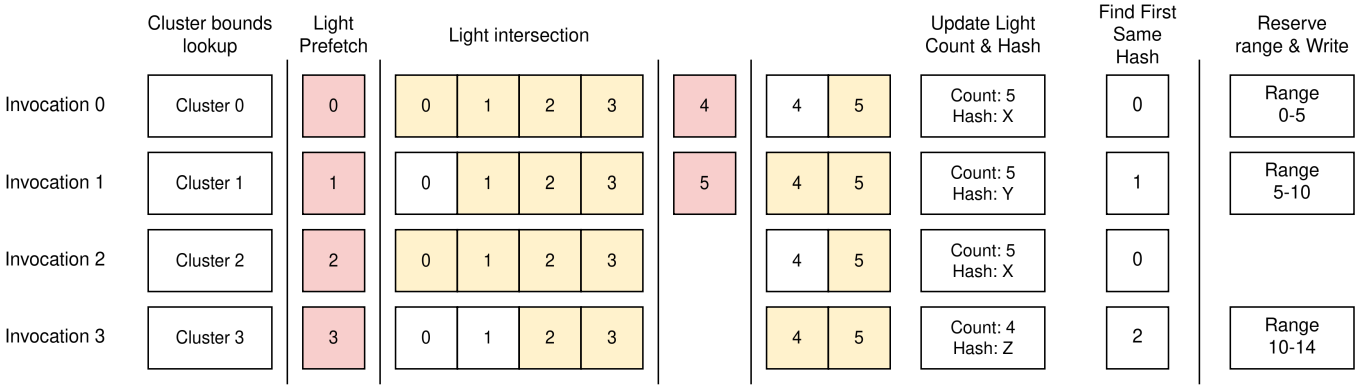

Figure 3: Optimized cluster assignment implementation. Vertical lines are barriers.

While much better, there are still some optimization opportunities. One idea is to cull lights early in the prefetching stage against the entire frustum.

IV.C. Forward Shading

Lastly, the cluster data can be used for shading. When shading a given fragment, we perform the opposite operation to the cluster generation, and get the cluster index of the view-space fragment position. We can then iterate over all lights in the cluster and shade the fragment using the lights.

Usually, multiple fragments in the same subgroup will be contained by the same cluster. This means that in theory, we should be able to reduce the amount of memory accesses and make them scalar. While this didn't seem to make a difference on NVIDIA,

V. Limitations

V.A. Reflections

The point lights influence is calculated using the range of diffuse lighting. Specular lighting has no upper range, and would require an influence range of infinity. This means that normal specular highlights no longer work. This means that reflections need to be handled in another way.

Luckily, reflections calculated using the lights is already very limited. Metallic surfaces usually become very dark, as there is rarely anything to reflect. This means that most engines already use another method for reflections. We suggest the following methods:

- Ray-traced reflections
- Screen space reflections
- IBL Cubemap reflections

Each method has its own advantages and disadvantages. Ray-traced reflections are the most accurate, but also likely the most expensive. Screen space reflections are cheaper, but have a problem with things outside the screen, requiring another method to fall back on. IBL Cubemap reflections are a good idea, as it synergizes well with the current clustering system, but has some PBR issues.

Cubemap reflections, hereforth called *Reflection Probes*, synergizes well with our clustering approach. Each cluster could track both lights and reflection probes, meaning that local reflections could be possible. One problem with reflection probes is that fragments not on the probe origin will be reflected incorrectly. This can be compensated for using parallax correction, but this doesn't fully solve it.

Bibliography

[1] Google, "google/shaderc: A collection of tools, libraries, and tests for Vulkan shader compilation.," GitHub. [Online]. Available: https://github.com/google/shaderc

[2] S. Han and P. V. Sander, "Triangle reordering for reduced overdraw in animated scenes," in *Proceedings of the 20th ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games*, 2016, pp. 23–27.

[3] O. Olsson, M. Billeter, and U. Assarsson, "Clustered deferred and forward shading," in *Proceedings of the Fourth ACM SIGGRAPH/Eurographics conference on High-Performance Graphics*, 2012, pp. 87–96.

[4] T. Sousa and J. Geffroy, "The Devil is in the Details," 2016.